1: Spring Data JPA – Quick Example

application.properties

```
logging.level.org.springframework=info
logging.level.com.cognizant=debug
logging.level.org.hibernate.SQL=trace
logging.level.org.hibernate.type.descriptor.sql=trace
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5Dialect
```

Country.java

```
package com.cognizant.ormlearn.model;

import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;
import javax.persistence.Column;

@Entity
@Table(name="country")
public class Country{
    @Id
    @Column(name="co_code")
    private String code;

    @Column(name="co_name")
    private String name;
```

```
    public String getCode(){
        return code;
    }

    public void setCode(String code){
        this.code=code;
    }

    public String getName(){
        return name;
    }

    public void setName(String name){
        this.name=name;
    }

    @Override
    public String toString(){
        return "Country [code="+code+", name="+name+"]";
    }
}
```

CountryRepository.java

```
package com.cognizant.ormlearn.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.cognizant.ormlearn.model.Country;
```

```
@Repository
public interface CountryRepository extends JpaRepository<Country,String>{
}
```

CountryService.java

```
package com.cognizant.ormlearn.service;

import java.util.List;
import javax.transaction.Transactional;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;

@Service
public class CountryService{
    @Autowired
    private CountryRepository countryRepository;

    @Transactional
    public List<Country> getAllCountries(){
        return countryRepository.findAll();
    }
}
```

OrmLearnApplication.java

```
package com.cognizant.ormlearn;

import java.util.List;
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;

@SpringBootApplication
public class OrmLearnApplication{
   private static final Logger
LOGGER=LoggerFactory.getLogger(OrmLearnApplication.class);
   private static CountryService countryService;

   public static void main(String[] args){
      ApplicationContext context=SpringApplication.run(OrmLearnApplication.class,args);
      countryService=context.getBean(CountryService.class);
      LOGGER.info("Inside main");
      testGetAllCountries();
   }

   private static void testGetAllCountries(){
      LOGGER.info("Start");
      List<Country> countries=countryService.getAllCountries();
      LOGGER.debug("Countries={}",countries);
      LOGGER.info("End");
   }
}
```

MySQL Table

```
CREATE TABLE country(
co_code VARCHAR(2) PRIMARY KEY,
co_name VARCHAR(50)
);

INSERT INTO country VALUES('IN','India');
INSERT INTO country VALUES('US','United States');
```

Output

```
Inside main
Start
Countries=[Country [code=IN, name=India], Country [code=US, name=United States]]
End
```

2:Difference between JPA, Hibernate and Spring Data JPA

Feature	JPA	Hibernate	Spring Data JPA
Type	Specification	ORM Framework	Spring Module
Implementation	No	Yes	No
Developed By	Oracle/Java Community	Hibernate Team	Spring
Boilerplate Code	More	Moderate	Very Less
CRUD Methods	Not Provided	Manual	Built-in (save(), findAll(), deleteById())
Transaction Management	API Only	Manual	Automatic using @Transactional

Feature	JPA	Hibernate	Spring Data JPA
Query Support	JPQL	HQL & SQL	Query Methods, JPQL, Native SQL
Dependency	Requires implementation	Independent ORM	Uses JPA implementation (Hibernate)

JPA

- JPA (Java Persistence API) is a **specification (JSR 338)**.
- It defines standards for Object Relational Mapping (ORM).
- It does **not provide an implementation**.
- Hibernate, EclipseLink, etc., implement JPA.

Hibernate

- Hibernate is an **ORM framework**.
- It is one of the most popular **implementations of JPA**.
- It maps Java objects to database tables.
- Requires more configuration than Spring Data JPA.

Spring Data JPA

- Spring Data JPA is a **Spring framework module** built on top of JPA.
- It reduces boilerplate code.
- Provides ready-made CRUD methods like:
    - save()
    - findAll()
    - findById()
    - deleteById()
- Uses Hibernate internally as the default JPA provider.

Conclusion

- **JPA** = Specification.
- **Hibernate** = Implementation of JPA.
- **Spring Data JPA** = Abstraction over JPA/Hibernate that simplifies database operations and minimizes code.